LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 10
MODEL DEPLOYMENT (FLASK)



Disusun oleh:

Nama	: Hafidz Rizqullah Prasetya
NIM	: 24/535493/SV/24243
Kelas	: PL5A1
Dosen Pengampu	: Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi **2**
Tujuan Praktikum **3**
Dasar Teori **4**
2.1 Model Deployment dalam Siklus Hidup Machine Learning 4
2.2 Alur Client-Server dan Metode HTTP 4
2.3 Flask 4
2.4 Serialisasi Model 5
2.5 Decision Tree 5
2.6 Support Vector Machine dan Kalibrasi Probabilitas 5
Alat dan Bahan **7**
3.1 Perangkat Keras 7
3.2 Perangkat Lunak 7
3.3 Dataset 7
Hasil dan Pembahasan **8**
4.1 Percobaan Modul: Deployment Model Klasifikasi Penyakit Hati 8
4.1.1 Persiapan Direktori dan Dataset 8
4.1.2 Pelatihan dan Serialisasi Model 9
4.1.3 Uji Deserialisasi dari Skrip 10
4.1.4 Web Service Flask dan Endpoint Prediksi 11
4.1.5 Menjalankan Server dan Pengujian 12
4.2 Tugas dan Analisis: Aplikasi Web GlucoSense 15
4.2.1 Pelatihan dan Pemilihan Model 15
4.2.2 Berkas Model dan Metadata 17
4.2.3 Rancangan Aplikasi Flask 17
4.2.4 Menjalankan Server 19
4.2.5 Pengujian Antarmuka Web 20
4.2.6 Pengujian Endpoint JSON 23
4.2.7 Analisis 23
Kesimpulan **24**

Tujuan Praktikum

Berdasarkan Modul Pertemuan 10 Praktikum Penambangan Data, tujuan praktikum ini adalah:

1. Memahami konsep dasar *model deployment* dalam siklus hidup *data mining* (*ML lifecycle*).
2. Melakukan *serialization* (penyimpanan) dan *deserialization* (pemuatan kembali) model *machine learning* dengan pustaka `joblib` atau `pickle`.
3. Membangun layanan web (*web service*) sederhana dengan *micro-framework* Flask.
4. Mengintegrasikan model *machine learning* ke antarmuka web dan API sehingga pengguna dapat mengirim data baru dan menerima hasil prediksi.

Dasar Teori

2.1 Model Deployment dalam Siklus Hidup Machine Learning

Pengembangan model *machine learning* berjalan sebagai siklus: perumusan masalah, pengumpulan data, penyiapan data, eksplorasi, rekayasa fitur, pelatihan, evaluasi, lalu *deployment* dan pemantauan [5]. Sampai Pertemuan 9, pekerjaan saya berhenti di tahap evaluasi. Model hanya hidup di dalam *notebook*, sehingga orang lain tidak bisa memakainya tanpa menjalankan ulang seluruh kode pelatihan.

Deployment adalah tahap yang membuat model bisa dipakai oleh pengguna atau sistem lain. Modul menyebut tiga skema: model yang ditanam langsung di aplikasi (*embedded*), prediksi berkala untuk banyak data sekaligus (*batch prediction*), dan arsitektur *client-server*. Praktikum ini memakai skema *client-server*. Model disimpan di server, sedangkan klien (peramban atau program lain) mengirim data lewat protokol HTTP dan menerima hasil prediksi sebagai halaman HTML atau JSON.

2.2 Alur Client-Server dan Metode HTTP

Pada skema *client-server*, satu permintaan prediksi melewati lima langkah. Klien membuka alamat (*endpoint*) yang disediakan server, lalu mengirim nilai fitur lewat form web atau badan permintaan JSON. Server mengurai dan memvalidasi masukan itu, kemudian menyusunnya menjadi bentuk yang diminta model. Model yang sudah dimuat ke memori menghitung prediksi, dan server mengirim hasilnya kembali ke klien.

Dua metode HTTP yang dipakai di praktikum ini adalah GET untuk meminta halaman dan POST untuk mengirim data. Server membalas dengan kode status, misalnya 200 untuk permintaan yang berhasil, 400 untuk masukan yang salah, 404 untuk alamat yang tidak ada, dan 500 untuk galat di sisi server. Gaya arsitektur yang memperlakukan setiap alamat sebagai sumber daya dan memakai metode HTTP apa adanya dikenal sebagai REST [4].

2.3 Flask

Flask adalah *micro web framework* berbahasa Python. Intinya hanya mengurus perutean (*routing*), objek permintaan dan balasan, serta *template* HTML lewat Jinja. Basis data dan validasi form tidak disertakan dan boleh ditambahkan sendiri bila perlu [9]. Sifat ini cocok untuk membungkus model, karena kode pelatihan dan kode server sama-sama Python sehingga objek model dapat langsung dipanggil di dalam fungsi *route*.

Sebuah *route* didefinisikan dengan dekorator `@app.route`. Nilai form dibaca dari `request.form`, badan JSON dibaca dengan `request.get_json()`, halaman dikembalikan dengan `render_template`, dan balasan JSON dibuat dengan `jsonify`. Perintah `app.run()` menjalankan server pengembangan bawaan Werkzeug. Dokumentasi Flask menyatakan server ini hanya untuk pengembangan; untuk produksi aplikasi dijalankan di belakang server WSGI seperti Gunicorn [10].

2.4 Serialisasi Model

Model yang selesai dilatih hanya ada di memori dan hilang ketika proses Python berakhir. *Serialization* mengubah objek model menjadi deretan *byte* yang disimpan sebagai berkas, dan *deserialization* memuat berkas itu kembali menjadi objek yang siap dipakai tanpa pelatihan ulang. Python menyediakan modul `pickle` untuk keperluan ini, sedangkan `joblib` memakai mekanisme yang sama tetapi lebih efisien untuk objek berisi *array* NumPy besar [7]. Fungsinya adalah `joblib.dump(model, berkas)` dan `joblib.load(berkas)`.

Ada dua hal yang perlu diingat. Pertama, memuat berkas `pickle` dapat menjalankan kode apa pun yang tersimpan di dalamnya, jadi berkas model hanya boleh dimuat dari sumber yang dipercaya [13]. Kedua, dokumentasi scikit-learn tidak menjamin model yang disimpan dengan satu versi dapat dimuat dengan versi lain, sehingga versi pustaka saat pelatihan sebaiknya dicatat bersama modelnya [14].

2.5 Decision Tree

Percobaan modul memakai `DecisionTreeClassifier`. Pohon keputusan membagi data secara berulang dengan satu pertanyaan pada satu fitur di setiap simpul. Scikit-learn mengimplementasikan algoritma CART [2, 11] dan secara bawaan memilih pembagian yang paling menurunkan ketidakmurnian Gini:

$$G = 1 - \sum_{k=1}^{K} p_k^2 \tag{1}$$

dengan p_k proporsi sampel kelas k pada simpul tersebut. Nilai $G = 0$ berarti simpul hanya berisi satu kelas.

2.6 Support Vector Machine dan Kalibrasi Probabilitas

Model terbaik pada bagian tugas adalah *Support Vector Machine* (SVM) dengan kernel *radial basis function* (RBF). SVM mencari bidang pemisah dengan margin terlebar di antara

dua kelas [3]. Kernel RBF mengukur kemiripan dua sampel dengan

$$K(x, x') = \exp\left(-\gamma \|x - x'\|^2\right) \tag{2}$$

sehingga batas keputusan boleh berbentuk tidak linier. Karena kernel ini bergantung pada jarak, setiap fitur distandarkan lebih dahulu dengan $z = (x - \mu)/\sigma$ agar fitur berskala besar seperti usia tidak mendominasi.

Keluaran asli SVM adalah jarak bertanda ke bidang pemisah, bukan probabilitas. Untuk mendapat peluang, jarak itu dipetakan dengan fungsi sigmoid yang parameternya dicari lewat validasi silang (metode Platt) [12]. Di scikit-learn, langkah ini dikerjakan oleh `CalibratedClassifierCV` [15].

Performa klasifikasi diukur dengan *accuracy*, *precision*, *recall*, dan *F1-score*:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{3}$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

Alat dan Bahan

3.1 Perangkat Keras

1. Laptop dengan prosesor 12 *thread* dan RAM 14 GB.
2. Koneksi internet untuk mengunduh berkas proyek modul.

3.2 Perangkat Lunak

1. Sistem operasi Linux Mint 22.3 64-bit.
2. Python 3.12.3 di dalam *virtual environment*.
3. Terminal dengan *shell* zsh untuk menjalankan skrip dan server.
4. Peramban Google Chrome 156 untuk menguji antarmuka web.
5. cURL 8.5.0 untuk menguji *endpoint* API.
6. Pustaka Python:
    - `flask` 3.1.3: layanan web.
    - `scikit-learn` 1.9.1: pelatihan dan evaluasi model.
    - `pandas` 3.0.5 dan `numpy` 2.5.3: pengolahan data tabular.
    - `joblib` 1.6.0: serialisasi model.

3.3 Dataset

1. Dataset HCV (`hcvdat0.csv`) dari berkas proyek modul [8], berisi 615 baris hasil uji laboratorium donor darah dan pasien hepatitis C dengan 12 fitur.
2. Dataset *Early Stage Diabetes Risk Prediction* [16, 6] yang dipakai proyek kelompok saya (Kelompok 3, GlucoSense), berisi 520 baris kuesioner pasien dengan 16 fitur.

Hasil dan Pembahasan

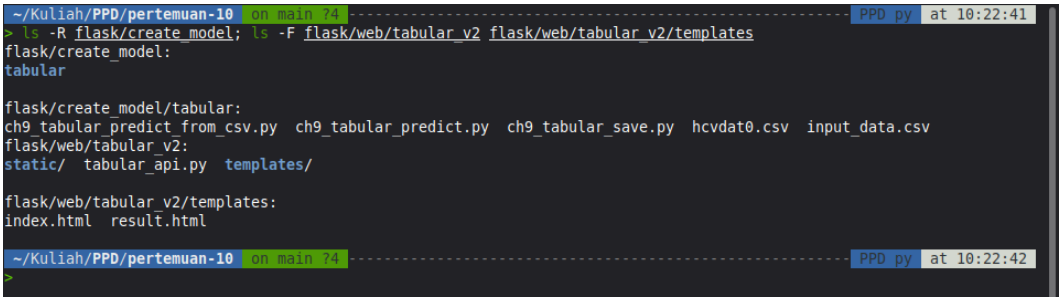
Seluruh percobaan saya jalankan langsung di laptop lewat terminal dan peramban, tanpa *notebook*, sesuai catatan modul bahwa server Flask tidak disarankan dijalankan di dalam Jupyter. Semua tangkapan layar pada bab ini diambil dari terminal dan peramban di laptop tersebut. Kode lengkap tersedia di repositori berikut:

https://github.com/hafidzrizqullahprasetya/PPD/tree/main/pertemuan-10 [1]

4.1 Percobaan Modul: Deployment Model Klasifikasi Penyakit Hati

4.1.1 Persiapan Direktori dan Dataset

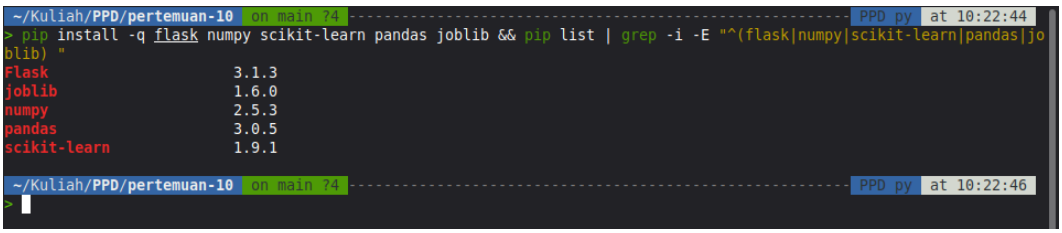
Berkas proyek saya unduh dari tautan Google Drive pada modul. Arsipnya berukuran sekitar 300 MB karena juga memuat contoh model citra, jadi saya hanya mengambil dua folder yang diminta modul, yaitu `flask/create_model/tabular` dan `flask/web/tabular_v2`.



Gambar 1: Isi folder pelatihan model dan folder aplikasi web dari berkas proyek modul.

Folder `tabular` berisi dataset `hcvdat0.csv` beserta tiga skrip: satu untuk melatih dan menyimpan model, dua untuk mencoba prediksi. Folder `tabular_v2` berisi skrip server `tabular_api.py`, folder `templates` untuk halaman HTML, dan folder `static` untuk berkas CSS dan JavaScript.

Pustaka saya pasang dengan perintah dari modul. Flask belum ada di *virtual environment* saya, sedangkan empat pustaka lain sudah terpasang sejak pertemuan sebelumnya.



Gambar 2: Pemasangan pustaka dan versi yang terpasang.

4.1.2 Pelatihan dan Serialisasi Model

Skrip pelatihan pada modul bernama `ch9_tabular_save.py`. Skrip ini membaca dataset, membuang baris yang memiliki nilai kosong, memisahkan fitur dari label `Category`, mengubah label teks menjadi angka 0 sampai 3, mengodekan kolom `Sex` dengan `LabelEncoder`, lalu melatih `DecisionTreeClassifier` dan menyimpannya dengan `joblib.dump`.

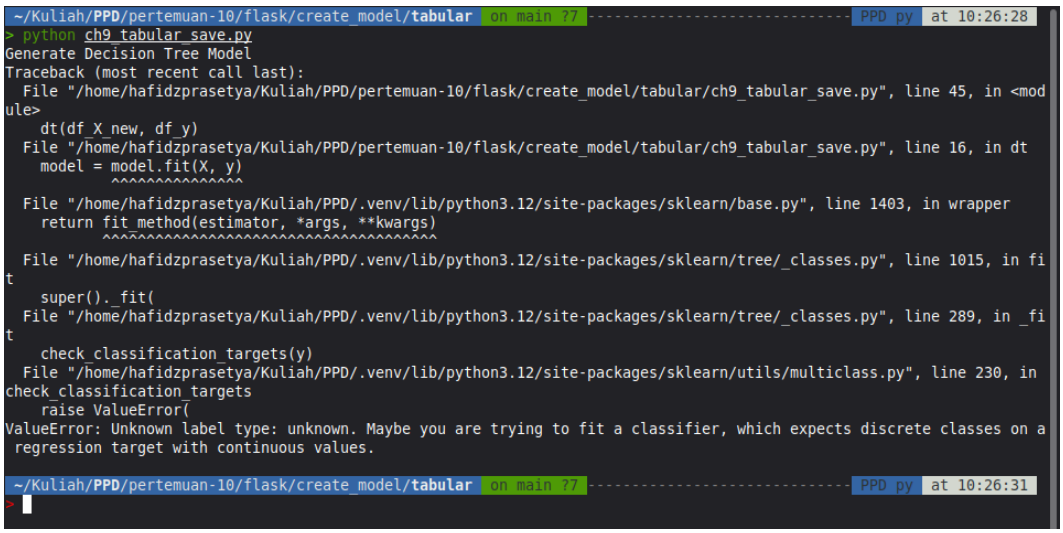
```
def dt(X, y):
    model = DecisionTreeClassifier(random_state=0)
    model = model.fit(X, y)
    filename = 'dt_model.model'
    joblib.dump(model, filename)

if __name__ == '__main__':
    df_ori = pd.read_csv('hcvdat0.csv')
    df_ori = df_ori.dropna()
    df_X = df_ori.drop(['Unnamed: 0','Category'],axis=1)
    df_y = df_ori['Category']
    df_y=df_y.replace('0=Blood Donor',0)
    df_y=df_y.replace('0s=suspect Blood Donor',0)
    df_y=df_y.replace('1=Hepatitis',1)
    df_y=df_y.replace('2=Fibrosis',2)
    df_y=df_y.replace('3=Cirrhosis',3)
    df_y = df_y.astype(int)       # baris tambahan saya

    df_X_numb = df_X.drop(['Sex'], axis=1)
    le = preprocessing.LabelEncoder()
    df_X['Sex'] = le.fit_transform(df_X['Sex'])
    df_X_new = pd.concat([df_X['Sex'], df_X_numb], axis=1)
    dt(df_X_new, df_y)
```

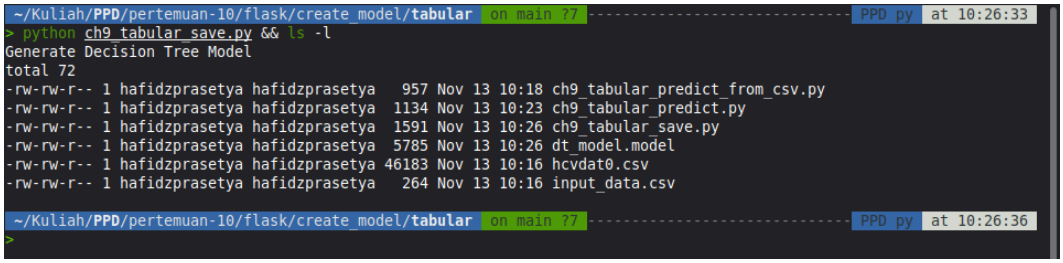
Listing 1: Inti skrip pelatihan dan serialisasi model (`ch9_tabular_save.py`).

Saat pertama kali saya jalankan apa adanya, skrip berhenti dengan galat seperti pada Gambar 3.



Gambar 3: Galat `Unknown label type` saat skrip modul dijalankan tanpa perubahan.

Penyebabnya ada di kolom label. Skrip modul ditulis tahun 2020, sedangkan saya memakai pandas 3.0.5. Di versi ini, kolom `Category` yang semula berisi teks tetap bertipe `object` setelah `replace`, walaupun isinya sudah angka 0 sampai 3. Scikit-learn tidak mengenali tipe itu sebagai label kelas sehingga menolak melatih model. Saya menambahkan satu baris `df_y = df_y.astype(int)` setelah proses pelabelan ulang, dan pelatihan berhasil.

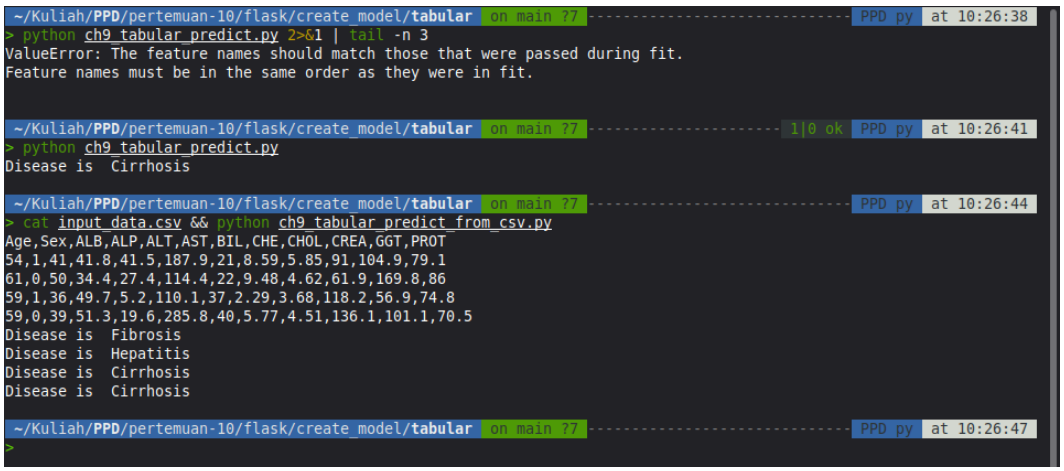


Gambar 4: Skrip pelatihan berhasil dan berkas `dt_model.model` terbentuk.

Berkas `dt_model.model` berukuran 5.785 *byte*. Dari 615 baris data, 589 baris tersisa setelah baris kosong dibuang. Pohon yang terbentuk memiliki kedalaman 6 dengan 22 daun. Ada dua catatan tentang model ini. Skrip modul melatih pohon pada seluruh data tanpa data uji, jadi tidak ada angka akurasi yang bisa saya laporkan, dan model ini memang hanya dipakai sebagai bahan latihan *deployment*. Selain itu kelasnya sangat timpang: 533 baris berlabel 0 (donor darah), sedangkan hepatitis 20 baris, fibrosis 12 baris, dan sirosis 24 baris.

4.1.3 Uji Deserialisasi dari Skrip

Sebelum masuk ke Flask, saya mencoba memuat kembali model dengan dua skrip prediksi dari modul. Skrip pertama membuat satu baris data baru di dalam kode, skrip kedua membaca empat baris dari `input_data.csv`.



Gambar 5: Galat urutan fitur pada skrip prediksi asli, lalu hasil prediksi setelah urutan kolom diperbaiki.

Skrip prediksi asli juga gagal, kali ini dengan pesan `Feature names must be in the same order as they were in fit`. Model dilatih dengan urutan kolom `Sex`, `Age`, `ALB`, dan seterusnya, karena skrip pelatihan menaruh kolom `Sex` paling depan lewat `pd.concat`. Skrip prediksi menyusun kolom dengan urutan `Age`, `Sex`, `ALB`. Scikit-learn versi sekarang menyimpan nama fitur saat `fit` dan menolak data dengan urutan berbeda. Menurut saya pemeriksaan ini sangat membantu. Tanpa itu, nilai usia akan terbaca sebagai jenis kelamin dan model tetap mengeluarkan prediksi yang salah tanpa pesan apa pun.

Perbaikannya saya lakukan di sisi data masukan. Pada skrip pertama urutan kolom saya ubah menjadi `Sex` dahulu. Pada skrip kedua saya menambahkan `df_input = df_input[loaded_model.feature_names_in_]` supaya urutan kolom CSV mengikuti urutan yang tersimpan di model. Setelah itu skrip pertama memprediksi *Cirrhosis*, dan empat baris CSV diprediksi *Fibrosis*, *Hepatitis*, *Cirrhosis*, dan *Cirrhosis*. Model yang dimuat dari berkas bekerja tanpa pelatihan ulang, jadi proses serialisasi dan deserialisasi berhasil.

4.1.4 Web Service Flask dan Endpoint Prediksi

Kode server pada modul ada di berkas `tabular_api.py` dan memiliki dua *route*. *Route* `/tabular/` menampilkan form `index.html`. *Route* `/tabular/result/` menerima kiriman form dengan metode POST, memuat model, menjalankan prediksi, lalu menampilkan `result.html`.

Versi `tabular_api.py` di dalam arsip unduhan masih menyusun kolom dengan urutan `Age` lalu `Sex`, sehingga terkena galat yang sama dengan skrip prediksi. Kode pada Gambar 10.4.10 modul sudah versi yang diperbaiki, jadi saya menyesuaikan berkas saya dengan kode di modul tersebut. Versi ini menambahkan dua fungsi bantu. Fungsi `_to_float` mengubah teks dari form menjadi bilangan, dan `_encode_sex` menerima isian seperti `m`, `male`, atau `1` lalu mengubahnya menjadi 1.

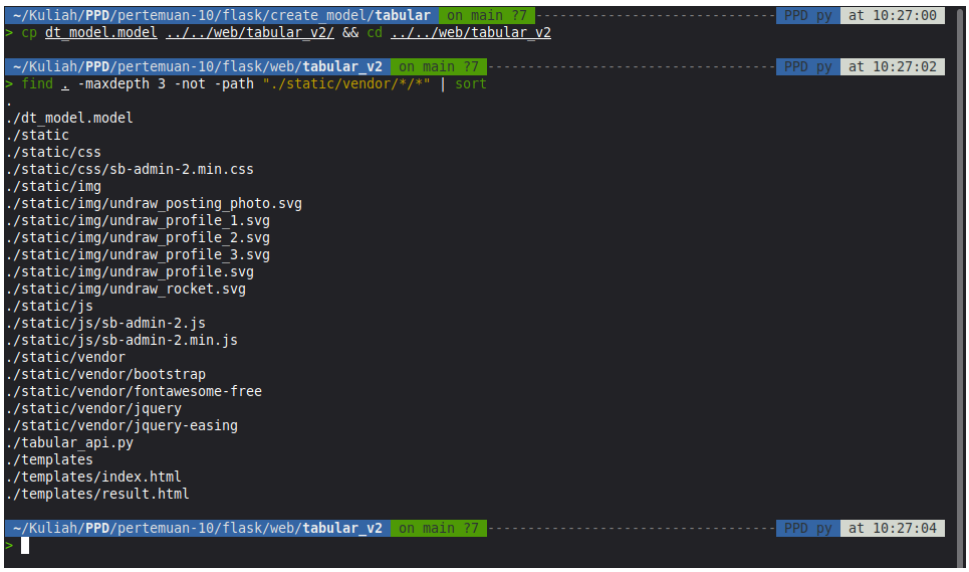
```python
app = Flask(__name__)

@app.route("/tabular/")
def index():
    return render_template('index.html')

@app.route('/tabular/result/', methods=["POST"])
def prediction_result():
    age = request.form.get('age')
    sex = request.form.get('sex')
    # ... sepuluh fitur lain dibaca dengan cara yang sama
    loaded_model = joblib.load('dt_model.model')
    sex_enc = _encode_sex(sex)
    columns_order = ['Sex', 'Age', 'ALB', 'ALP', 'ALT', 'AST',
                     'BIL', 'CHE', 'CHOL', 'CREA', 'GGT', 'PROT']
    df_input = pd.DataFrame(columns=columns_order)
    df_input.loc[0] = [sex_enc, _to_float(age), _to_float(alb), ...]
    result = loaded_model.predict(df_input)
    # ... angka 0 sampai 3 dipetakan ke nama penyakit
    return render_template('result.html', age=age, sex=sex, ...,
                           status=decision)
```

Listing 2: Potongan *endpoint* prediksi pada `tabular_api.py`.

Berkas `dt_model.model` kemudian saya salin ke folder `tabular_v2`, karena server mencari berkas model di folder tempat ia dijalankan.

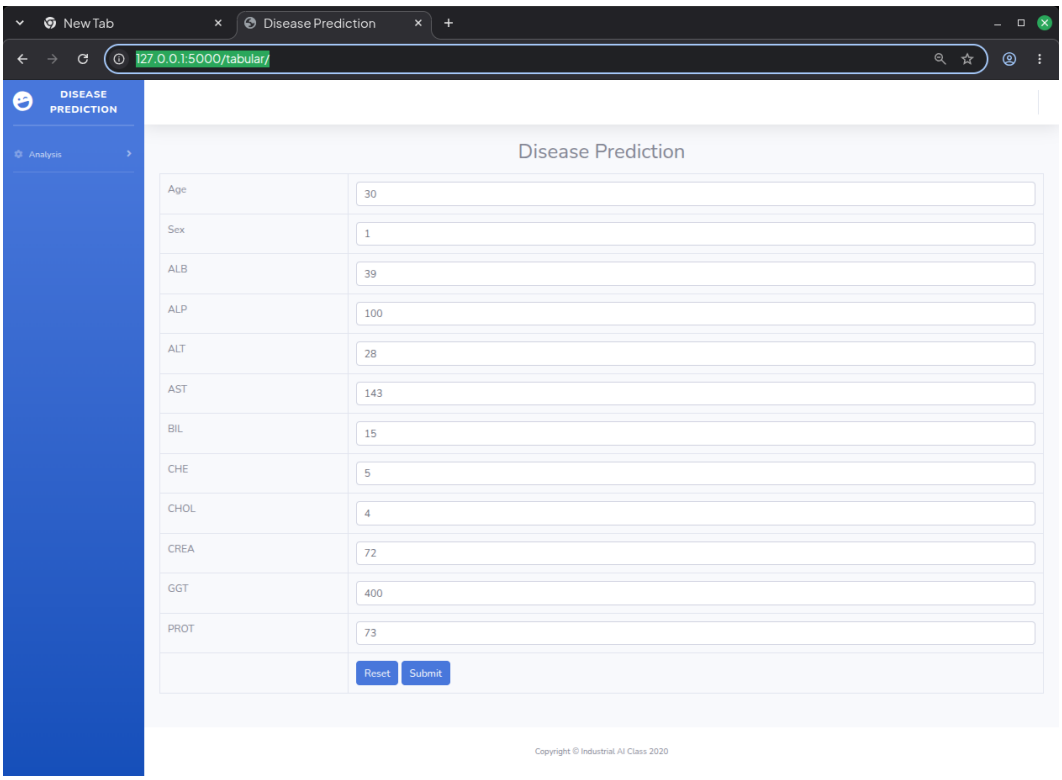


Gambar 6: Penyalinan model dan struktur proyek aplikasi Flask.

Struktur ini mengikuti konvensi Flask. Halaman HTML harus berada di folder `templates` agar ditemukan oleh `render_template`, dan berkas CSS, JavaScript, serta ikon berada di folder `static` yang otomatis dilayani pada alamat `/static/`.

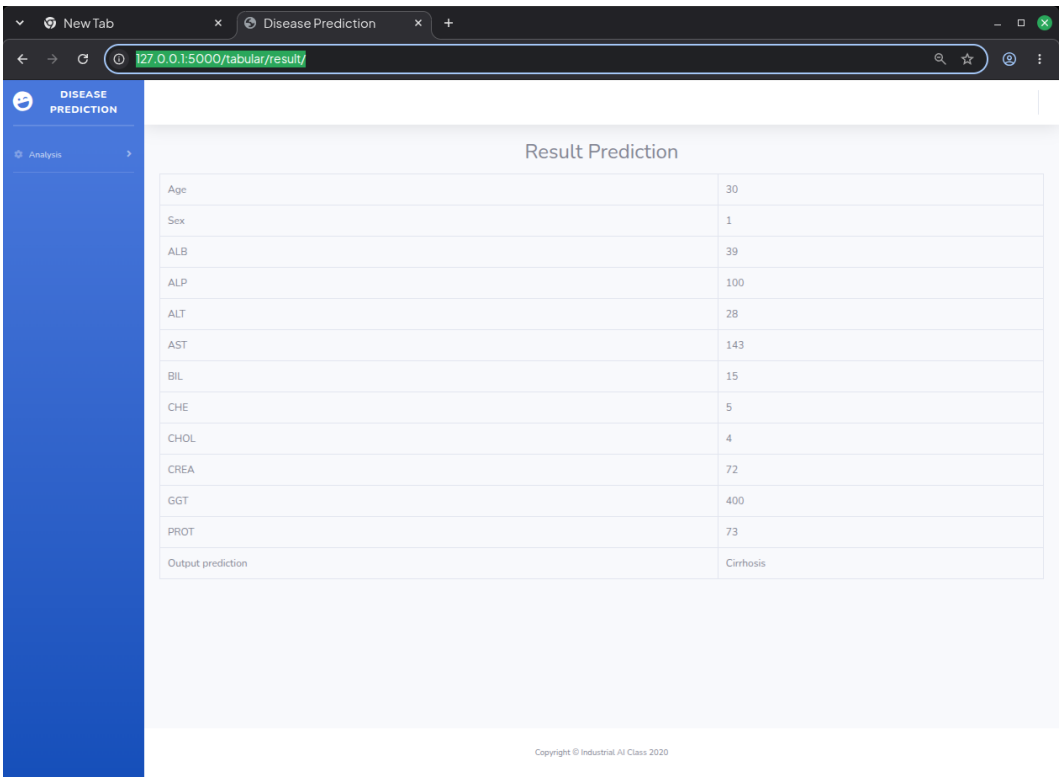
4.1.5 Menjalankan Server dan Pengujian

Server saya jalankan dengan `python tabular_api.py`, lalu saya buka `http://127.0.0.1:5000/tabular/` di peramban.



Gambar 7: Form masukan data klinis pada *endpoint* `/tabular/`.

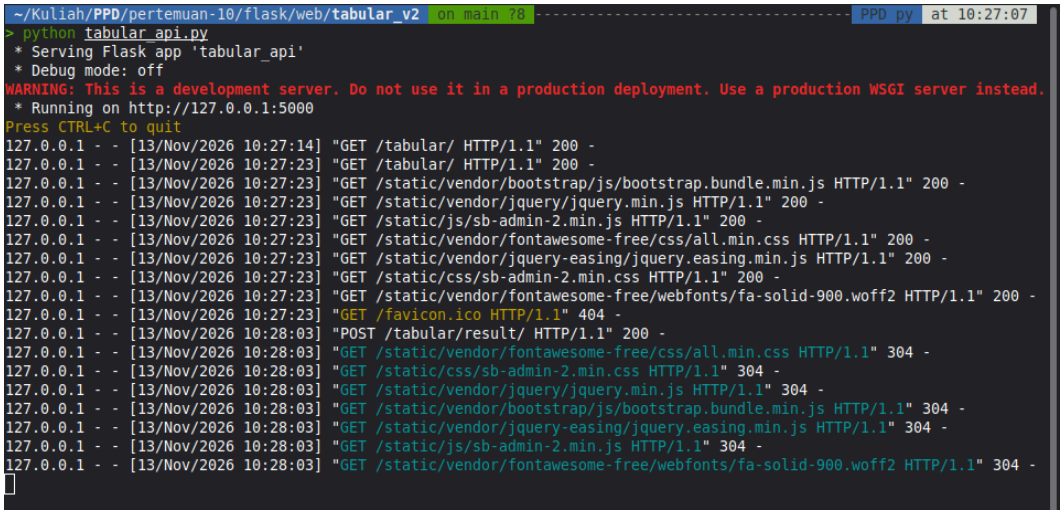
Form menampilkan 12 kolom isian dengan nilai bawaan dari *template*. Nilai bawaan itu saya kirim dengan menekan tombol *Submit*.



Gambar 8: Halaman hasil prediksi pada *endpoint* `/tabular/result/`.

Halaman hasil menampilkan kembali 12 nilai yang dikirim dan prediksi *Cirrhosis* pada

baris *Output prediction*. Nilai AST 143 dan GGT 400 pada isian ini memang jauh di atas kebanyakan donor sehat di dataset, jadi prediksi sirosis masuk akal.



Gambar 9: Log server Flask modul selama pengujian.

Log server pada Gambar 9 memperlihatkan alur *client-server* yang dijelaskan di dasar teori. Permintaan `GET /tabular/` dibalas 200, lalu peramban meminta berkas CSS dan JavaScript dari `/static/`. Saat tombol *Submit* ditekan, muncul baris `POST /tabular/result/` dengan status 200. Kode 304 pada permintaan berikutnya berarti peramban memakai salinan berkas statis yang sudah tersimpan, dan 404 untuk `favicon.ico` muncul karena aplikasi ini tidak punya ikon. Baris peringatan berwarna merah mengingatkan bahwa server bawaan ini hanya untuk pengembangan.

Dari percobaan modul ini saya mencatat dua kelemahan pada aplikasinya. Model dimuat ulang dengan `joblib.load` di setiap permintaan, dan form menerima teks bebas tanpa validasi. Kolom *Sex* misalnya harus diisi angka 1 atau 0, tanpa keterangan mana yang laki-laki. Keduanya saya perbaiki pada aplikasi tugas.

4.2 Tugas dan Analisis: Aplikasi Web GlucoSense

Tugas modul meminta dataset proyek kelompok dipakai untuk melatih model, model terbaiknya diserialisasi, lalu dibuatkan antarmuka web dengan Flask. Proyek Kelompok 3 bernama GlucoSense, yaitu deteksi dini risiko diabetes dari gejala klinis awal. Dataset berisi usia, jenis kelamin, dan 14 gejala berjawaban ya atau tidak, misalnya sering buang air kecil (*polyuria*) dan sering haus (*polydipsia*), dengan label *Positive* atau *Negative*. Bagian saya di kelompok adalah pemodelan dan *backend* prediksi, jadi tugas ini sekaligus menjadi rancangan awal untuk bagian tersebut.

4.2.1 Pelatihan dan Pemilihan Model

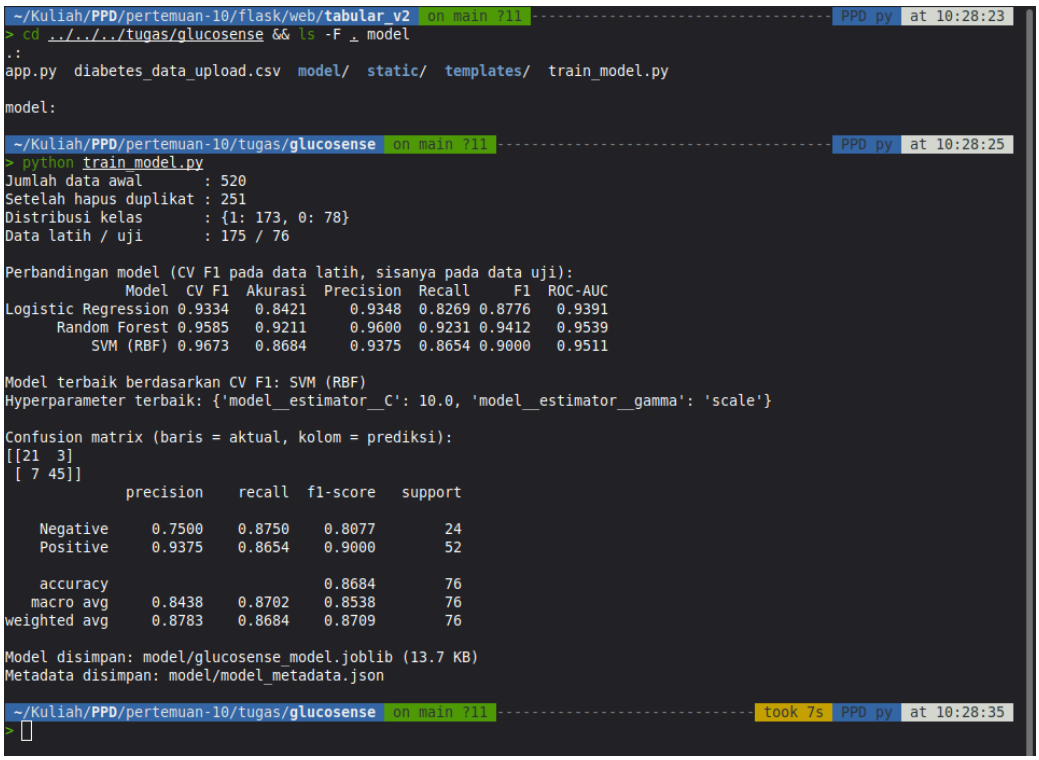
Skrip `train_model.py` mengikuti prapemrosesan yang sudah disepakati kelompok: baris duplikat dihapus, jawaban *Yes/No* diubah menjadi 1/0, *Male/Female* menjadi 1/0, dan data dibagi 70% latih dan 30% uji secara terstratifikasi. Bedanya dengan *notebook* kelompok, penskalaan dan model saya gabungkan dalam satu objek `Pipeline`.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
hasil, terbaik = [], None
for nama, (estimator, grid) in kandidat_model().items():
    pipa = Pipeline([('scaler', StandardScaler()), ('model', estimator)])
    cari = GridSearchCV(pipa, grid, cv=cv, scoring='f1', n_jobs=-1)
    cari.fit(X_train, y_train)
    ...
    if terbaik is None or cari.best_score_ > terbaik[1].best_score_:
        terbaik = (nama, cari)

nama, cari = terbaik
model = cari.best_estimator_
joblib.dump(model, MODEL_DIR / 'glucosense_model.joblib')
```

Listing 3: Pencarian *hyperparameter* dan pemilihan model pada `train_model.py`.

Ada tiga kandidat, yaitu *Logistic Regression*, *Random Forest*, dan SVM berkernel RBF. Masing-masing dicari *hyperparameter* terbaiknya dengan `GridSearchCV` dan validasi silang 5 lipatan pada data latih. Model dipilih dari rata-rata F1 validasi silang, bukan dari skor data uji. Untuk SVM saya membungkusnya dengan `CalibratedClassifierCV`, karena parameter `probability=True` pada `SVC` sudah berstatus *deprecated* di scikit-learn 1.9 dan aplikasi web membutuhkan nilai peluang.



Gambar 10: Keluaran `train_model.py`: perbandingan model, model terpilih, dan evaluasinya.

Dari 520 baris, 269 baris ternyata duplikat sehingga tersisa 251 pasien, terdiri atas 173 positif dan 78 negatif. Data latih berisi 175 baris dan data uji 76 baris. Tabel 1 merangkum hasil ketiga kandidat.

Tabel 1: Perbandingan tiga kandidat model pada dataset diabetes.

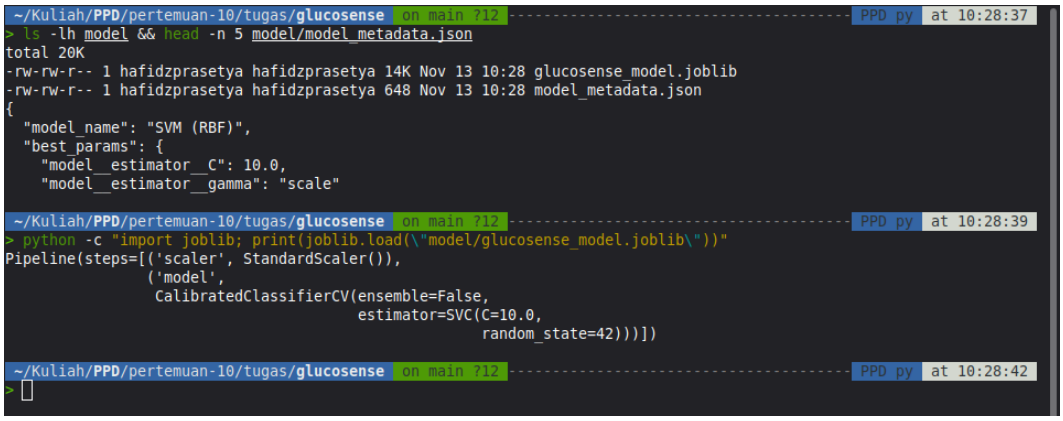
Model	**F1 CV**	**Akurasi**	**Precision**	**Recall**	**F1**	**ROC-AUC**
Logistic Regression	0,9334	0,8421	0,9348	0,8269	0,8776	0,9391
Random Forest	0,9585	**0,9211**	**0,9600**	**0,9231**	**0,9412**	**0,9539**
SVM (RBF)	**0,9673**	0,8684	0,9375	0,8654	0,9000	0,9511

SVM memiliki F1 validasi silang tertinggi (0,9673) dengan $C = 10$ dan γ bawaan, sehingga model inilah yang disimpan. Hasil di data uji ternyata berbeda: *Random Forest* lebih baik dengan akurasi 0,9211, sedangkan SVM 0,8684. Saya tetap menyimpan SVM. Kalau model dipilih karena angka ujinya paling tinggi, data uji ikut dipakai untuk mengambil keputusan dan angkanya tidak lagi mewakili data yang belum pernah dilihat. Selisih kedua model juga kecil dibanding ukuran datanya. Di data uji selisih itu setara 4 dari 76 pasien, dan selisih F1 validasi silang hanya 0,0088. Dengan data sesedikit ini saya belum bisa menyimpulkan salah satu model benar-benar lebih unggul. Pilihan final untuk proyek kelompok masih perlu dibahas bersama anggota lain, misalnya dengan validasi silang berulang.

Pada *confusion matrix* SVM, 45 dari 52 pasien positif terdeteksi dan 7 pasien positif

diprediksi negatif (*false negative*). Sebaliknya, 3 dari 24 pasien negatif diprediksi positif. Untuk alat skrining, 7 kasus yang lolos itu lebih merugikan daripada 3 alarm palsu, karena pasien yang sebenarnya berisiko bisa merasa aman. *Recall* 0,8654 ini menjadi kelemahan utama model saat ini.

4.2.2 Berkas Model dan Metadata



Gambar 11: Berkas hasil serialisasi dan isi `Pipeline` saat dimuat kembali.

Pelatihan menghasilkan dua berkas, yaitu model `glucosense_model.joblib` sebesar 14 KB dan `model_metadata.json`. Saat dimuat kembali dengan `joblib.load`, objeknya adalah `Pipeline` berisi `StandardScaler` dan SVM terkalibrasi. Pada proyek kelompok, *scaler* dan model disimpan sebagai dua berkas terpisah, sehingga aplikasi web harus ingat memanggil keduanya dengan urutan yang benar. Dengan satu `Pipeline`, aplikasi cukup memanggil `predict` dan penskalaan ikut berjalan.

Berkas metadata menyimpan nama model, *hyperparameter* terbaik, urutan nama fitur, jumlah data, skor, dan versi scikit-learn. Urutan fitur saya simpan karena pengalaman galat pada percobaan modul, dan versi pustaka saya catat mengikuti saran dokumentasi scikit-learn [14].

4.2.3 Rancangan Aplikasi Flask

Aplikasi `app.py` memiliki tiga *endpoint* seperti pada Tabel 2. Selain form web seperti pada modul, saya menambahkan satu *endpoint* JSON agar model juga bisa dipanggil oleh program lain, misalnya aplikasi web kelompok nanti.

Tabel 2: *Endpoint* aplikasi GlucoSense.

Metode	**Alamat**	**Fungsi**
GET	`/`	Menampilkan form skrining berisi usia, jenis kelamin, dan 14 gejala.
POST	`/predict`	Menerima kiriman form, lalu menampilkan halaman hasil atau form beserta pesan galat.
POST	`/api/predict`	Menerima JSON dan membalas JSON berisi kelas, label, dan peluang positif.

```python
# Deserialisasi dilakukan sekali saat server mulai, bukan di setiap request.
model = joblib.load(MODEL_PATH)
with open(METADATA_PATH) as f:
    metadata = json.load(f)
FITUR = metadata['feature_names']

def prediksi(nilai):
    # Urutan kolom diambil dari metadata agar sama dengan saat pelatihan.
    df_input = pd.DataFrame([nilai], columns=FITUR)
    kelas = int(model.predict(df_input)[0])
    peluang = float(model.predict_proba(df_input)[0][1])
    return {'kelas': kelas,
            'label': metadata['target_classes'][str(kelas)],
            'peluang_positif': round(peluang, 4)}

@app.route('/predict', methods=['POST'])
def predict():
    nilai, galat = validasi(request.form)
    if galat:
        return render_template('index.html', gejala=GEJALA,
                               isian=request.form, galat=galat,
                               meta=metadata), 400
    hasil = prediksi(nilai)
    ...

@app.route('/api/predict', methods=['POST'])
def api_predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify(error='Body request harus berupa objek JSON.'), 400
    nilai, galat = validasi(data)
    if galat:
        return jsonify(error='Input tidak valid.', detail=galat), 400
    return jsonify(model=metadata['model_name'], input=nilai,
                   **prediksi(nilai))
```

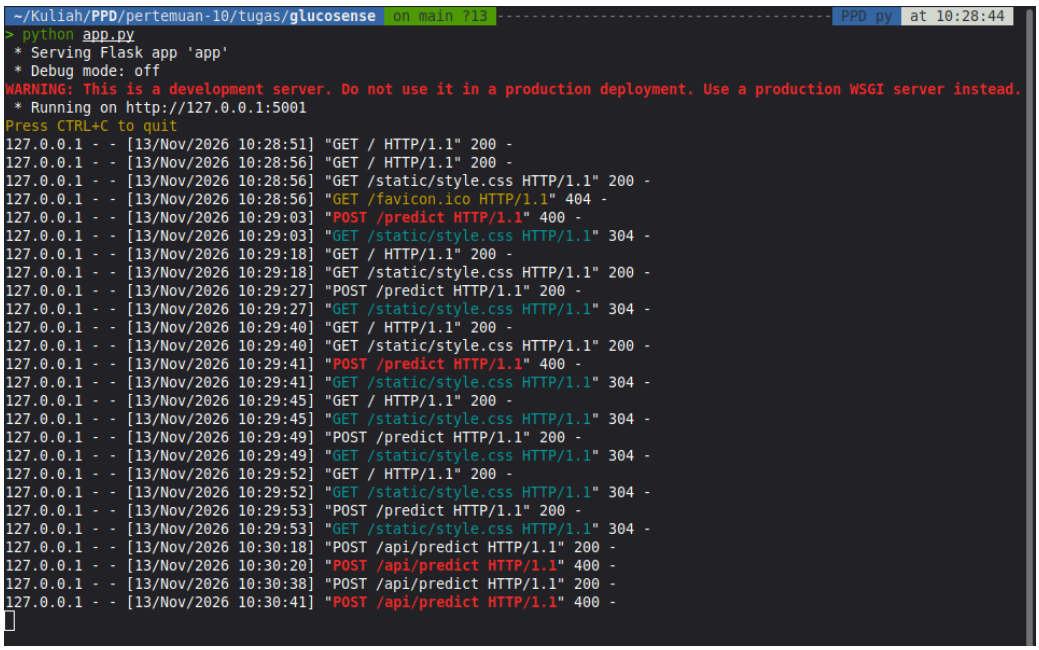
Listing 4: Pemuatan model, fungsi inferensi, dan *endpoint* pada `app.py`.

Ada tiga keputusan rancangan yang saya ambil dari pengalaman percobaan modul. Model dimuat satu kali saat server dinyalakan, sehingga setiap permintaan tidak perlu membaca berkas dari diska. Urutan kolom diambil dari metadata, bukan diketik ulang di kode server. Kedua *endpoint* prediksi memakai fungsi `validasi` dan `prediksi` yang sama, sehingga form web dan API selalu memberi hasil yang sama untuk masukan yang sama. Fungsi `validasi` memeriksa bahwa usia berupa bilangan bulat 1 sampai 120, jenis kelamin terisi, dan semua gejala dijawab. Kalau ada yang salah, server membalas dengan status 400 beserta

daftar pesannya.

4.2.4 Menjalankan Server

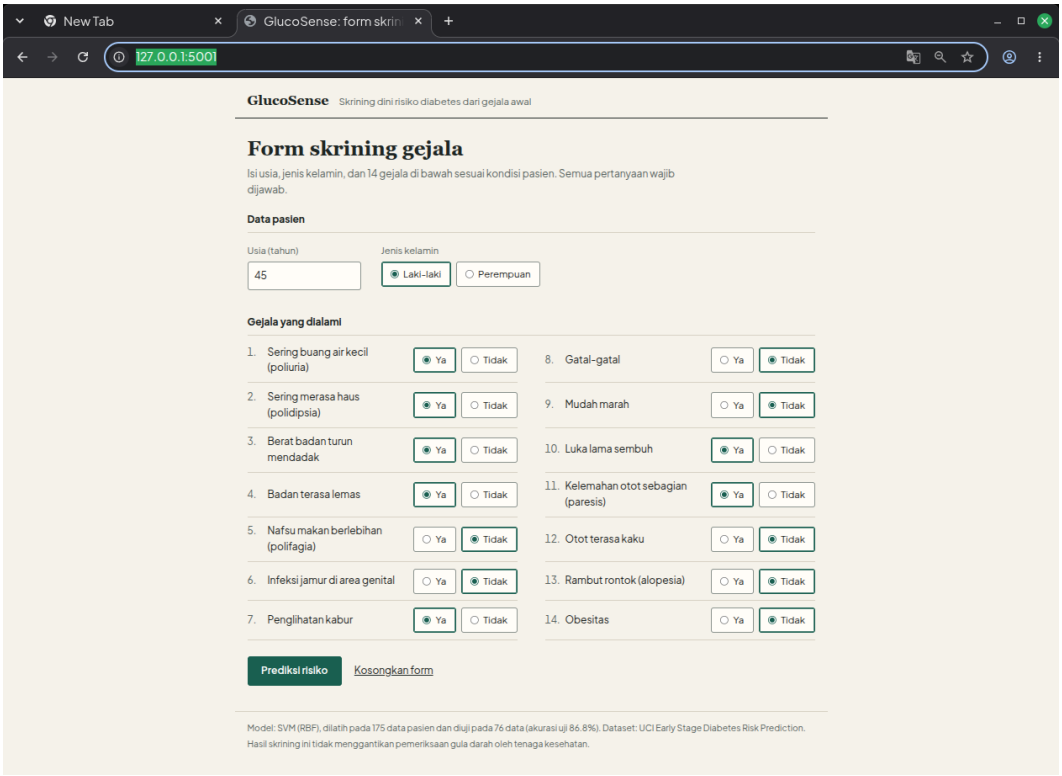
Aplikasi saya jalankan di port 5001 dengan `python app.py`.



Gambar 12: Log server GlucoSense selama seluruh pengujian.

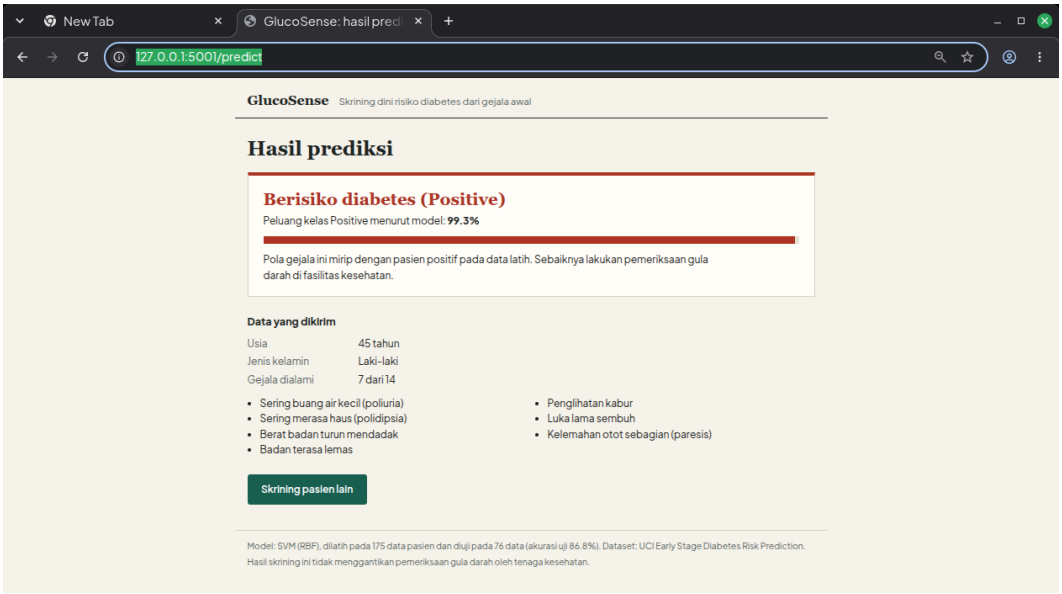
Log pada Gambar 12 merekam semua pengujian di subbab berikutnya. Baris `POST /predict` berstatus 400 berasal dari uji validasi form, baris berstatus 200 dari prediksi yang berhasil, dan dua baris terakhir dari uji `/api/predict` dengan cURL.

4.2.5 Pengujian Antarmuka Web



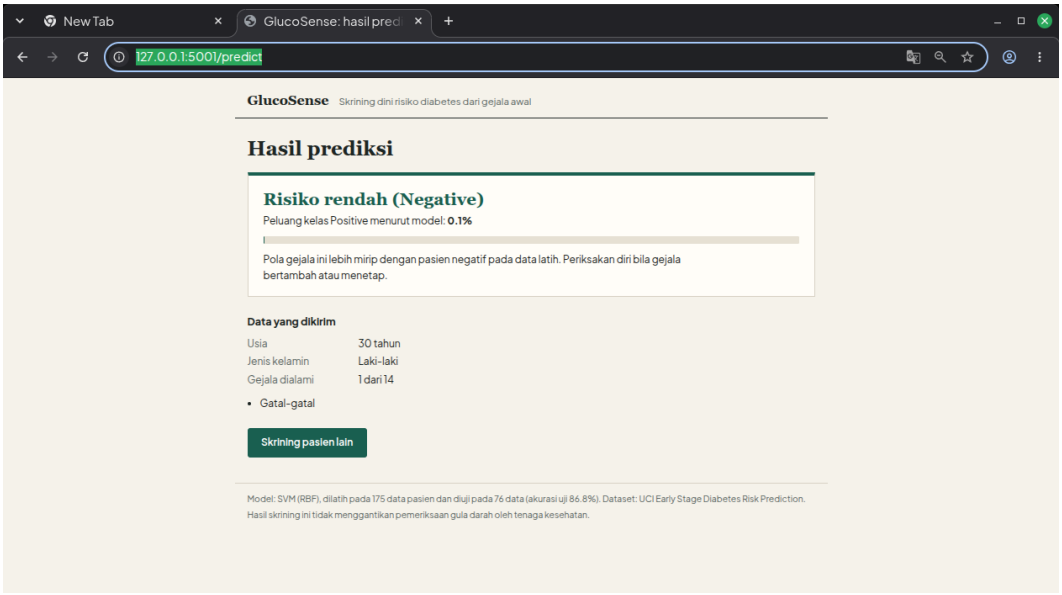
Gambar 13: Form skrining GlucoSense yang sudah diisi data pasien uji.

Form menampilkan gejala dalam bahasa Indonesia dengan pilihan Ya dan Tidak, sehingga pengguna tidak perlu tahu bahwa model sebenarnya menerima angka 1 dan 0. Pasien uji pertama saya isi sebagai laki-laki berusia 45 tahun dengan tujuh gejala, termasuk sering buang air kecil, sering haus, dan berat badan turun mendadak.



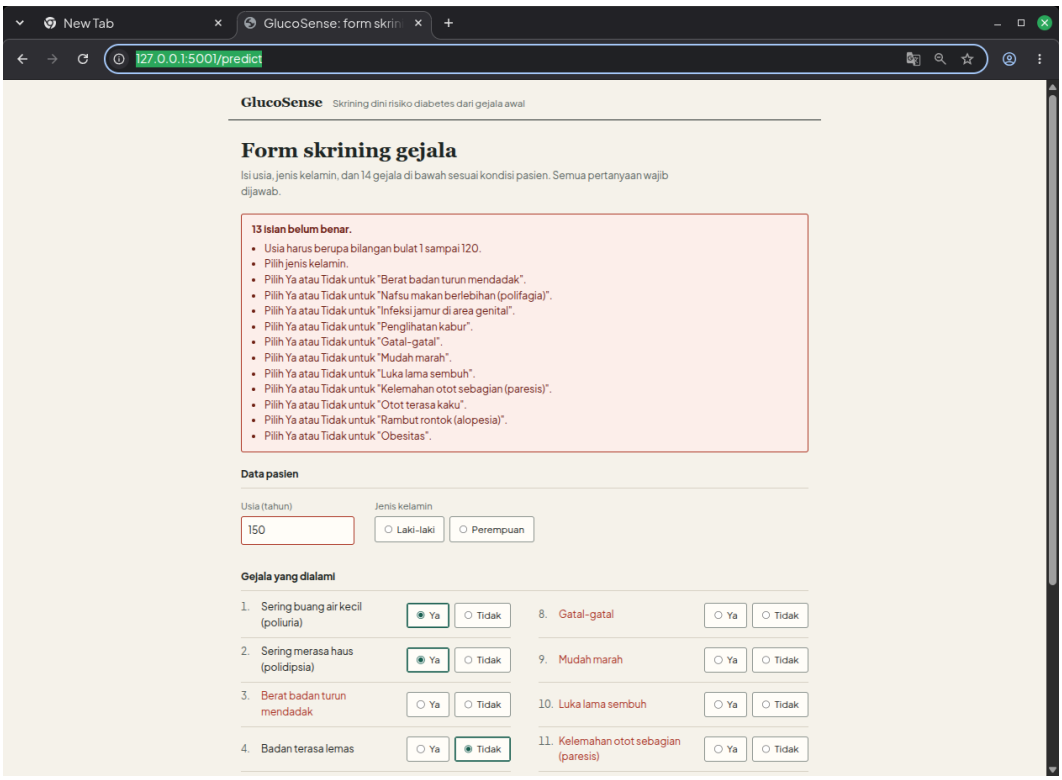
Gambar 14: Halaman hasil untuk pasien uji pertama: prediksi *Positive*.

Model memprediksi pasien ini *Positive* dengan peluang 99,3%. Halaman hasil juga menampilkan ringkasan data yang dikirim dan daftar gejala yang dijawab Ya, supaya pengguna bisa memeriksa ulang isiannya.



Gambar 15: Halaman hasil untuk pasien uji kedua: prediksi *Negative*.

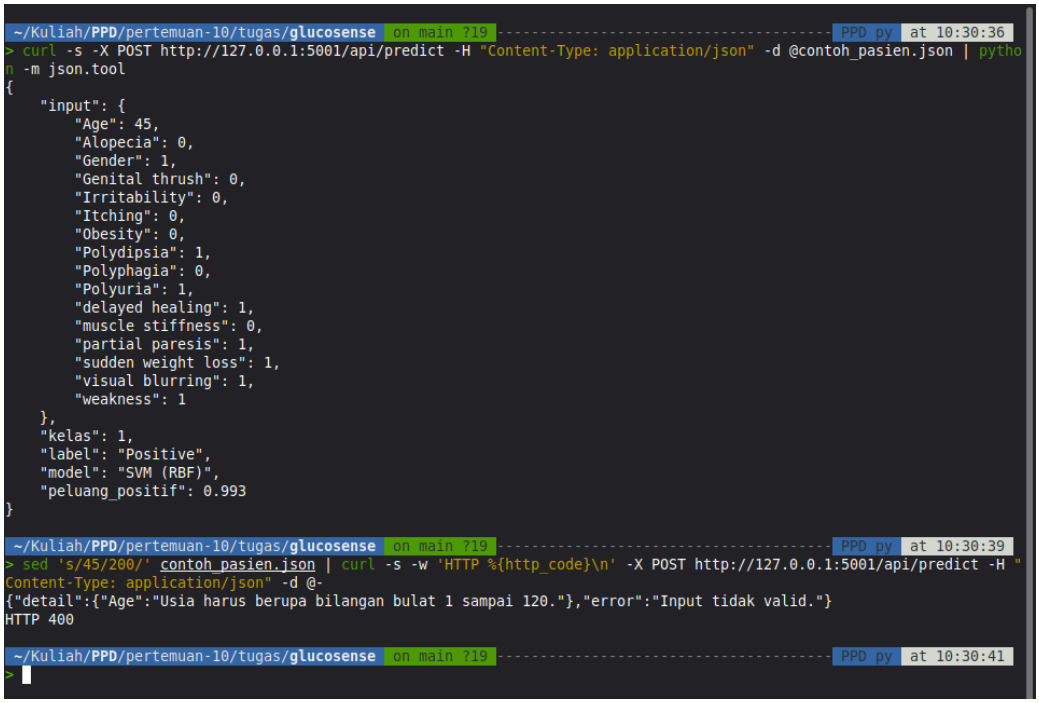
Pasien uji kedua adalah laki-laki berusia 30 tahun yang hanya mengalami gatal-gatal. Model memprediksi *Negative* dengan peluang positif 0,1%. Kedua hasil ini sejalan dengan temuan kelompok saya bahwa *polyuria* dan *polydipsia* adalah gejala yang paling kuat membedakan kedua kelas. Angka peluang yang sangat dekat dengan 0% dan 100% perlu dibaca hati-hati. Model hanya dilatih dengan 175 pasien dari satu rumah sakit, jadi angka itu menggambarkan seberapa mirip isian dengan data latih dan belum tentu mewakili peluang klinis yang sebenarnya.



Gambar 16: Pesan validasi saat usia diisi 150 dan sebagian pertanyaan dikosongkan.

Untuk menguji validasi, saya mengisi usia 150 dan hanya menjawab tiga gejala. Server menolak kiriman itu dengan status 400 dan menampilkan 13 pesan galat. Tiga jawaban yang sudah diisi tetap terpilih, sehingga pengguna tidak perlu mengulang dari awal. Tanpa validasi ini, isian kosong akan sampai ke model dan menimbulkan galat 500 atau prediksi yang tidak bermakna.

4.2.6 Pengujian Endpoint JSON



Gambar 17: Pengujian `/api/predict` dengan cURL untuk masukan valid dan tidak valid.

Data pasien uji pertama saya simpan sebagai `contoh_pasien.json` lalu saya kirim dengan cURL. Balasannya berupa JSON berisi masukan yang sudah dikodekan menjadi angka, `kelas` 1, `label` *Positive*, dan `peluang_positif` 0,993. Angka ini sama dengan yang tampil di halaman web untuk pasien yang sama. Pada percobaan kedua, usia saya ganti menjadi 200. Server membalas dengan status 400 dan pesan bahwa usia harus berupa bilangan bulat 1 sampai 120.

4.2.7 Analisis

Dibanding aplikasi modul, ada empat perbedaan pada GlucoSense. Model dimuat sekali saat server mulai. Penskalaan dan model berada dalam satu berkas. Masukan divalidasi sebelum sampai ke model. Selain halaman HTML, tersedia balasan JSON untuk program lain.

Aplikasi ini masih punya beberapa keterbatasan. Server yang dipakai masih server pengembangan Flask dan hanya bisa diakses dari laptop saya sendiri. Supaya bisa dipakai orang lain, aplikasi perlu dijalankan dengan server WSGI seperti Gunicorn dan dipasang di layanan *hosting* [10]. Berkas model juga terikat pada scikit-learn 1.9.1, sehingga lingkungan server harus memakai versi yang sama. Dari sisi model, *recall* 0,8654 berarti sekitar satu dari tujuh pasien positif di data uji tidak terdeteksi. Salah satu perbaikan yang bisa dicoba adalah menurunkan ambang peluang di bawah 0,5 agar lebih banyak kasus positif tertangkap, dengan konsekuensi alarm palsu bertambah.

Kesimpulan

Berdasarkan percobaan modul dan tugas yang saya kerjakan pada Pertemuan 10, kesimpulannya adalah sebagai berikut:

1. *Deployment* dengan skema *client-server* membuat model bisa dipakai lewat peramban tanpa menjalankan ulang kode pelatihan. Pada percobaan modul, model *Decision Tree* penyakit hati berhasil dilayani Flask dan memprediksi *Cirrhosis* untuk isian bawaan form.
2. Serialisasi dengan `joblib.dump` menghasilkan berkas model berukuran 5.785 *byte* untuk *Decision Tree* dan 14 KB untuk `Pipeline` SVM. Keduanya dapat dimuat kembali dengan `joblib.load` dan langsung dipakai untuk prediksi.
3. Kode modul yang ditulis tahun 2020 perlu dua penyesuaian di pustaka versi sekarang: label kelas harus diubah menjadi bilangan bulat, dan urutan kolom masukan harus sama dengan urutan saat pelatihan. Karena itu urutan fitur dan versi pustaka sebaiknya disimpan bersama model.
4. Pada dataset diabetes (251 pasien setelah duplikat dihapus), SVM berkernel RBF terpilih dengan F1 validasi silang 0,9673. Di data uji akurasinya 0,8684 dan F1 0,9000, sedangkan *Random Forest* mencapai akurasi 0,9211. Selisih ini setara 4 dari 76 pasien, jadi belum cukup untuk menyatakan salah satu model lebih unggul.
5. Aplikasi GlucoSense melayani prediksi lewat form web dan *endpoint* JSON dengan fungsi inferensi yang sama. Pasien uji dengan tujuh gejala diprediksi *Positive* (peluang 99,3%), pasien dengan satu gejala diprediksi *Negative* (0,1%), dan masukan yang salah ditolak dengan status 400.
6. Aplikasi masih berjalan di server pengembangan Flask pada laptop sendiri. Untuk dipakai orang lain, aplikasi perlu server WSGI dan *hosting*, dan *recall* model (0,8654) perlu dinaikkan karena kasus positif yang lolos adalah kesalahan paling merugikan pada alat skrining.

Daftar Pustaka

[1] H. R. Prasetya, "Praktikum 10: Model Deployment (Flask)," GitHub, 2026. [Online]. Tersedia: https://github.com/hafidzrizqullahprasetya/PPD/tree/main/pertemuan-10.

[2] L. Breiman, J. H. Friedman, R. A. Olshen, dan C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[3] C. Cortes dan V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, hlm. 273 sampai 297, 1995.

[4] R. T. Fielding, "Architectural styles and the design of network-based software architectures," disertasi doktor, University of California, Irvine, 2000. [Online]. Tersedia: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm.

[5] J. Han, J. Pei, dan H. Tong, *Data Mining: Concepts and Techniques*, edisi ke-4. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[6] M. M. F. Islam, R. Ferdousi, S. Rahman, dan H. Y. Bushra, "Likelihood prediction of diabetes at early stage using data mining techniques," dalam *Computer Vision and Machine Intelligence in Medical Image Analysis*. Singapore: Springer, 2020, hlm. 113 sampai 125.

[7] Joblib developers, "Persistence," Joblib Documentation. [Online]. Tersedia: https://joblib.readthedocs.io/en/stable/persistence.html.

[8] R. Lichtinghagen, F. Klawonn, dan G. Hoffmann, "HCV data," UCI Machine Learning Repository, 2020. [Online]. Tersedia: https://doi.org/10.24432/C5D612.

[9] Pallets, "Quickstart," Flask Documentation. [Online]. Tersedia: https://flask.palletsprojects.com/en/stable/quickstart/.

[10] Pallets, "Deploying to production," Flask Documentation. [Online]. Tersedia: https://flask.palletsprojects.com/en/stable/deploying/.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, hlm. 2825 sampai 2830, 2011.

[12] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," dalam *Advances in Large Margin Classifiers*. Cambridge, MA, USA: MIT Press, 1999, hlm. 61 sampai 74.

[13] Python Software Foundation, “pickle: Python object serialization,” Python 3 Documentation. [Online]. Tersedia: https://docs.python.org/3/library/pickle.html.

[14] Scikit-learn developers, “Model persistence,” Scikit-learn Documentation. [Online]. Tersedia: https://scikit-learn.org/stable/model_persistence.html.

[15] Scikit-learn developers, “Probability calibration,” Scikit-learn Documentation. [Online]. Tersedia: https://scikit-learn.org/stable/modules/calibration.html.

[16] UCI Machine Learning Repository, “Early stage diabetes risk prediction dataset,” 2020. [Online]. Tersedia: https://doi.org/10.24432/C5VG8H.